AI-Powered Bulk Document Scanning system

- Samuel Mugisha D.C

Agenda

- Executive Summary
- Business Problem
- Exploratory Data Analysis
- Models
- Model Performance Summary
- Deployment
- Tools used & Skills demonstrated
- Future improvements
- Conclusion

Executive Summary

- Developed a computer vision model capable of detecting whether a document page is currently being flipped or not, using a single image frame.

- Developed models: **VGG-16 Base, VGG-16 Base + FFNN, VGG-16 Base + FFNN + Data Augmentation, Efficient V2B1, ResNet50, MobileNetV2**

- The final MobileNetV2 solution achieved 89.6% F1 Score, coupled with its light-weight suitable for Mobile environments.

Business challenge

How can we automatically detect page flips to enable high-quality document scanning in bulk for users including the blind and researchers?

Key application: Fully automatic, highly fast and high-quality document scanning in bulk.

Business context

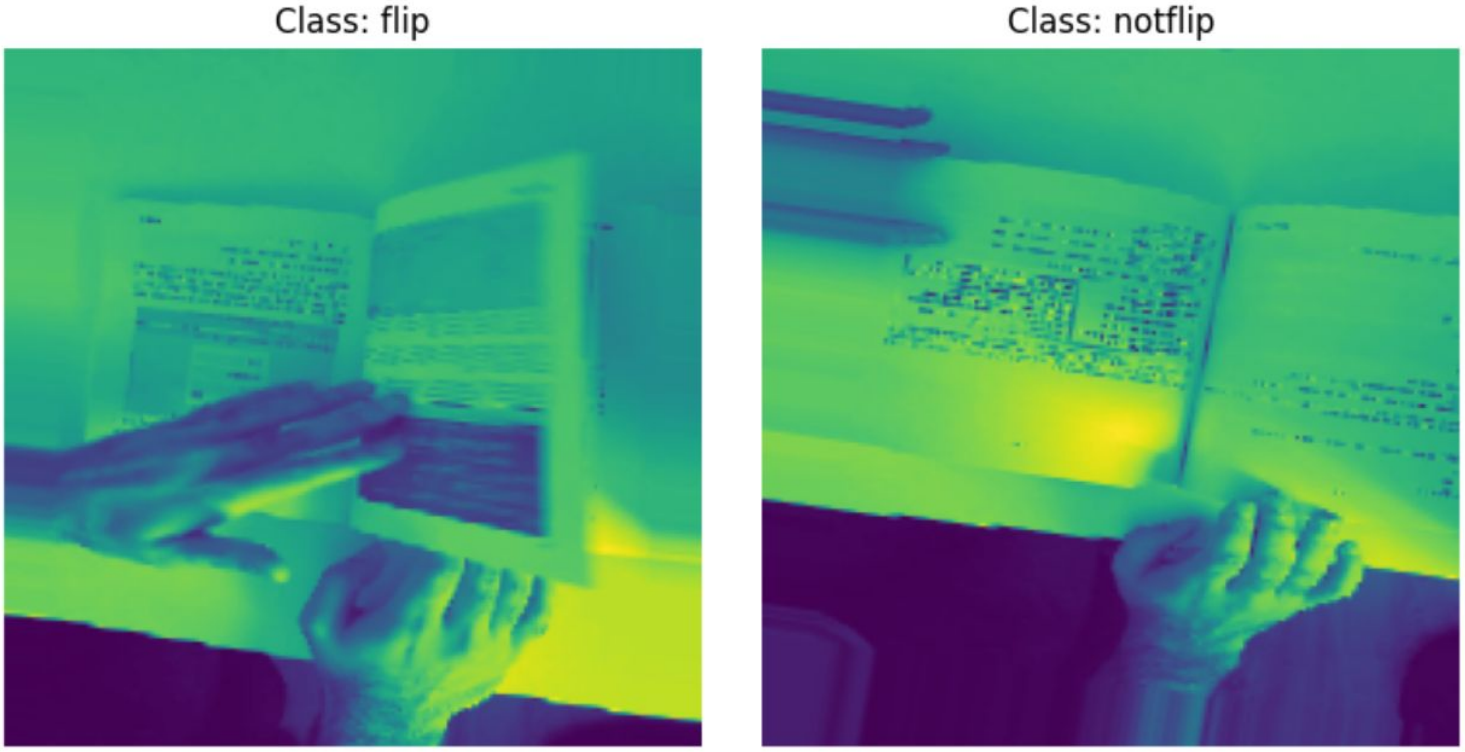
MonReader Mobile app:

- **detects page flips from low-resolution camera preview and takes a high-resolution picture of the document,**
- **recognizing its corners and crops it accordingly, and it dewarps the cropped document to obtain a bird's eye view**
- **sharpens the contrast between the text and the background and finally recognizes the text with formatting kept intact, being further corrected by MonReader's ML powered redactor.**

Dataset:

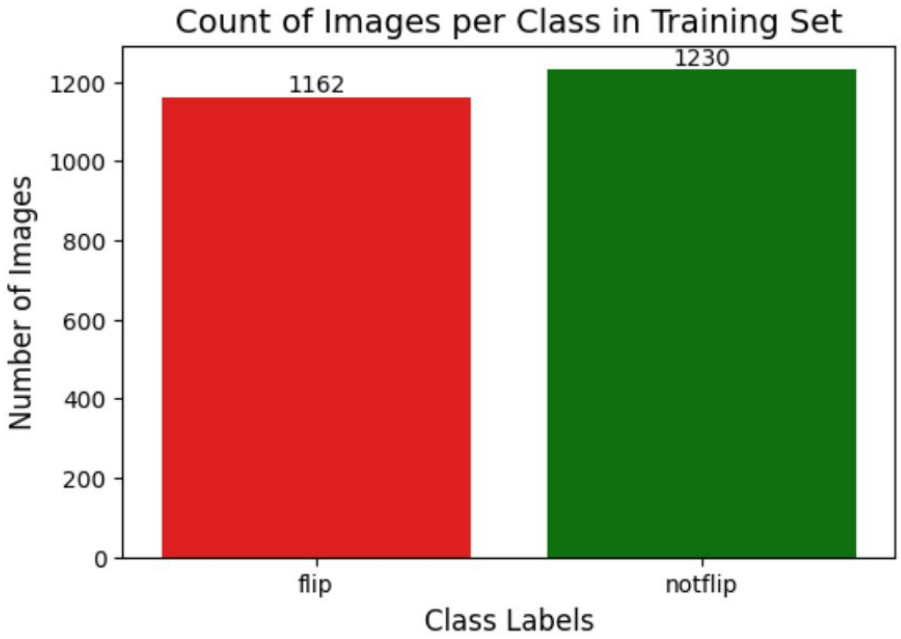
Dataset	Samples
Training Images	2,392
Testing Images	597
Image Size	200 × 200
Classes	2

EDA: Image Overview



- Plot random images from each of the classes and their corresponding labels.

EDA: Class balance




```
Class distribution in training set:
     Class  Count
0     flip   1162
1  notflip   1230
```

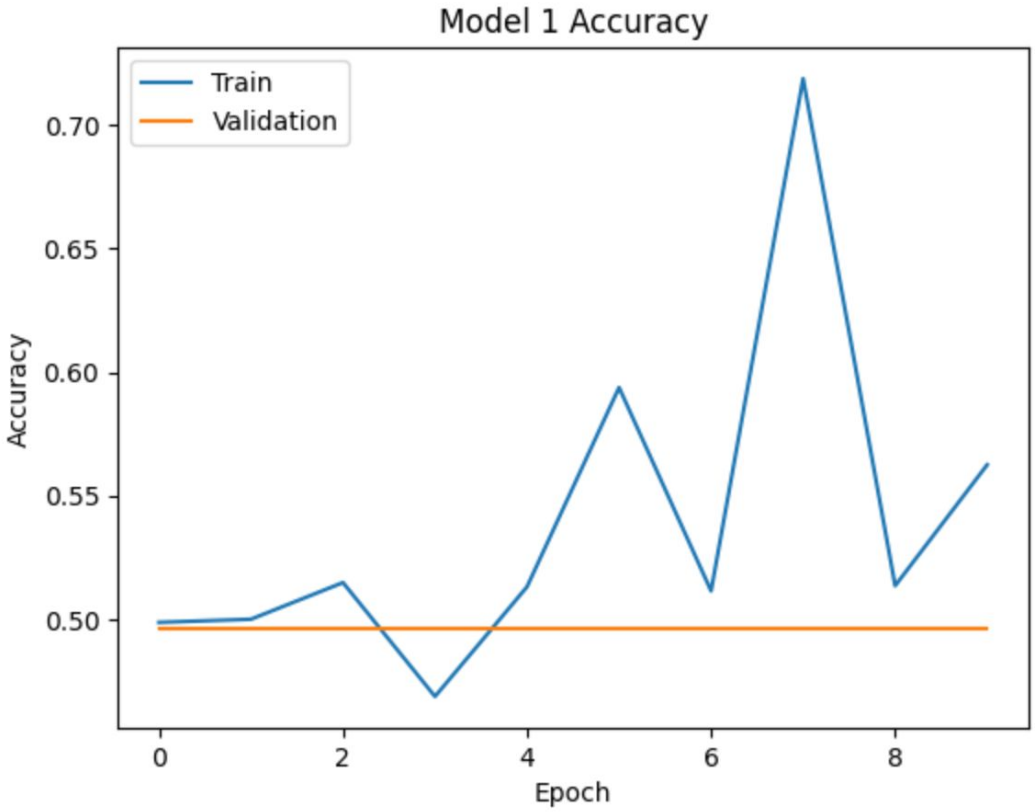
- The classes are quite balanced.

Data preprocessing

- Converting images to grayscale
- Splitting the data into
- Data normalisation

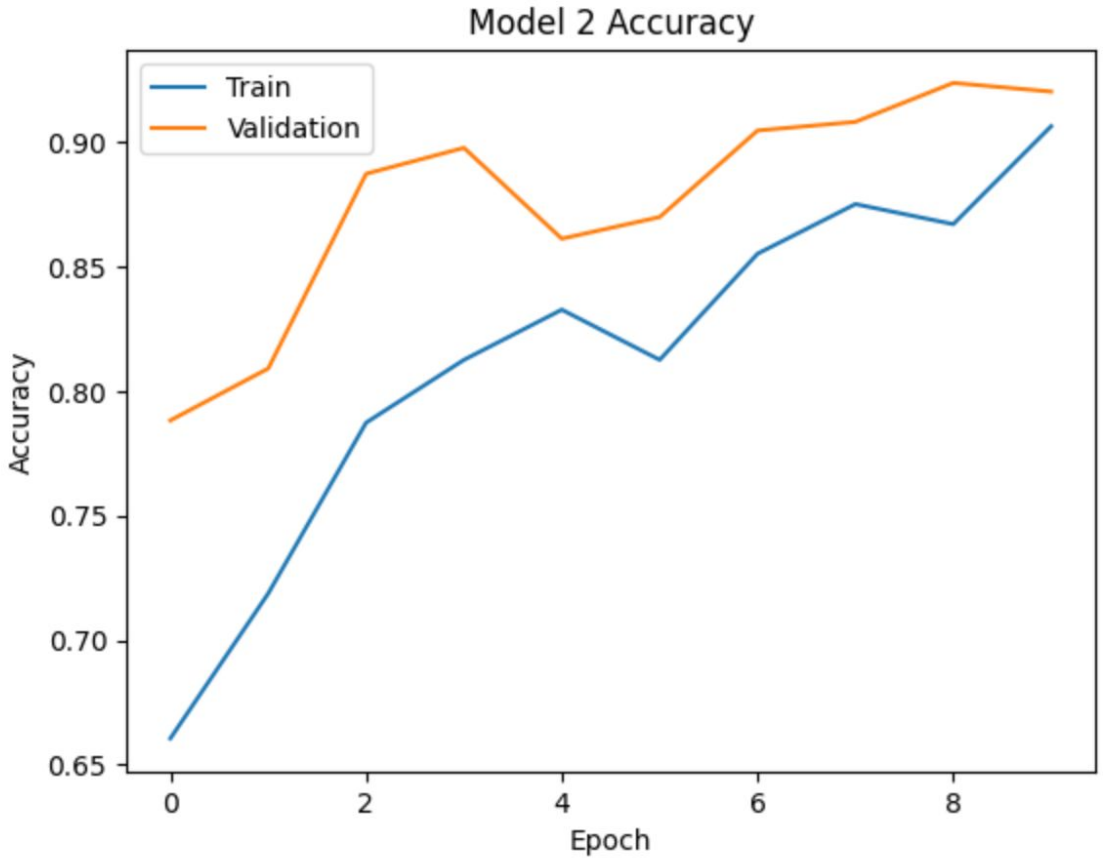
Results

Model 1: Simple CNN



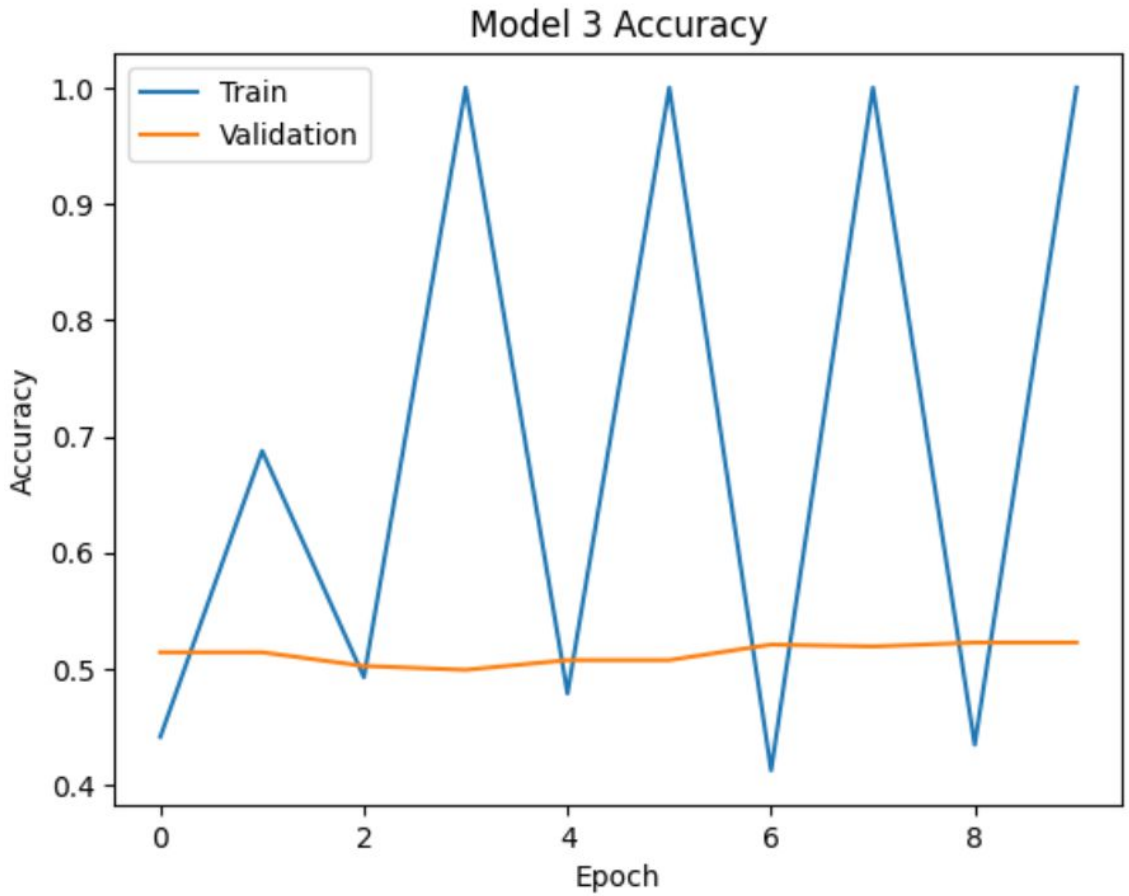
- **Both the training and validation accuracy remained stagnant and close to 50% throughout all epochs. This indicates that Model 1 was not able to learn any meaningful patterns from the data and performed no better than random chance. Essentially, the model failed to classify the 'flip' and 'notflip' images.**

Model 2: VGG16



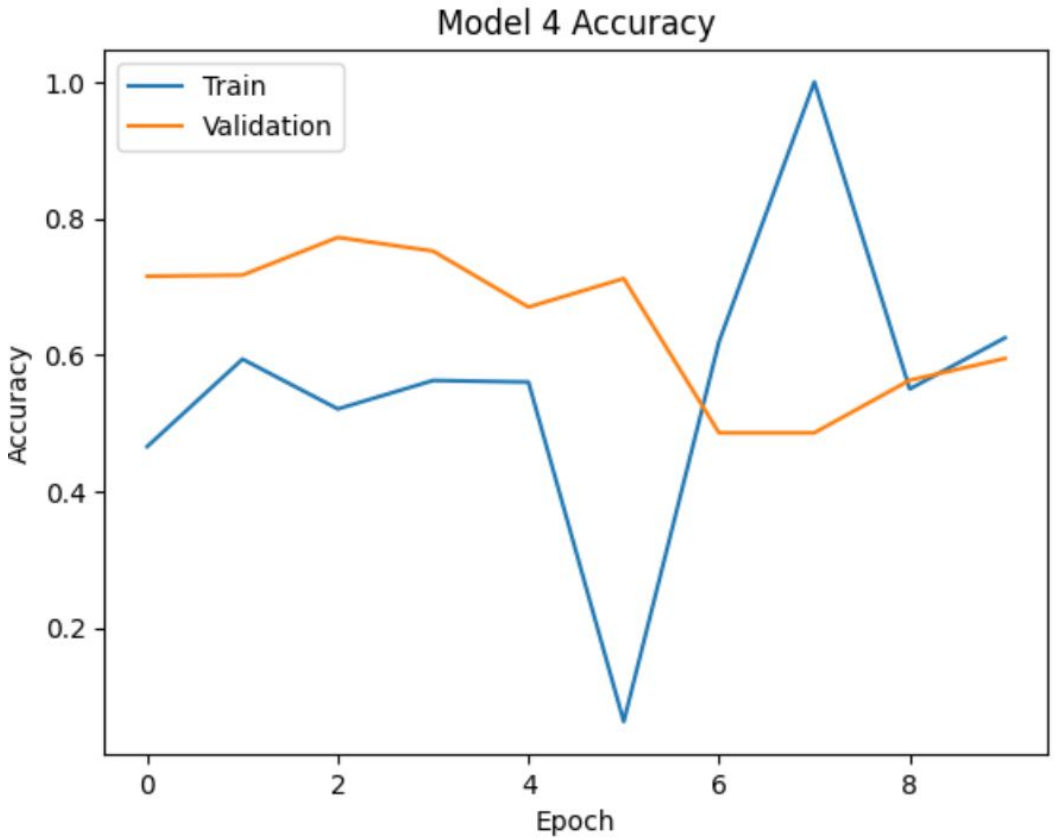
- A robust and reliable option with excellent performance (~90.43%) if computational resources are available and a slightly larger model size is acceptable.

Model 3: VGG16 (Base + FFNN)



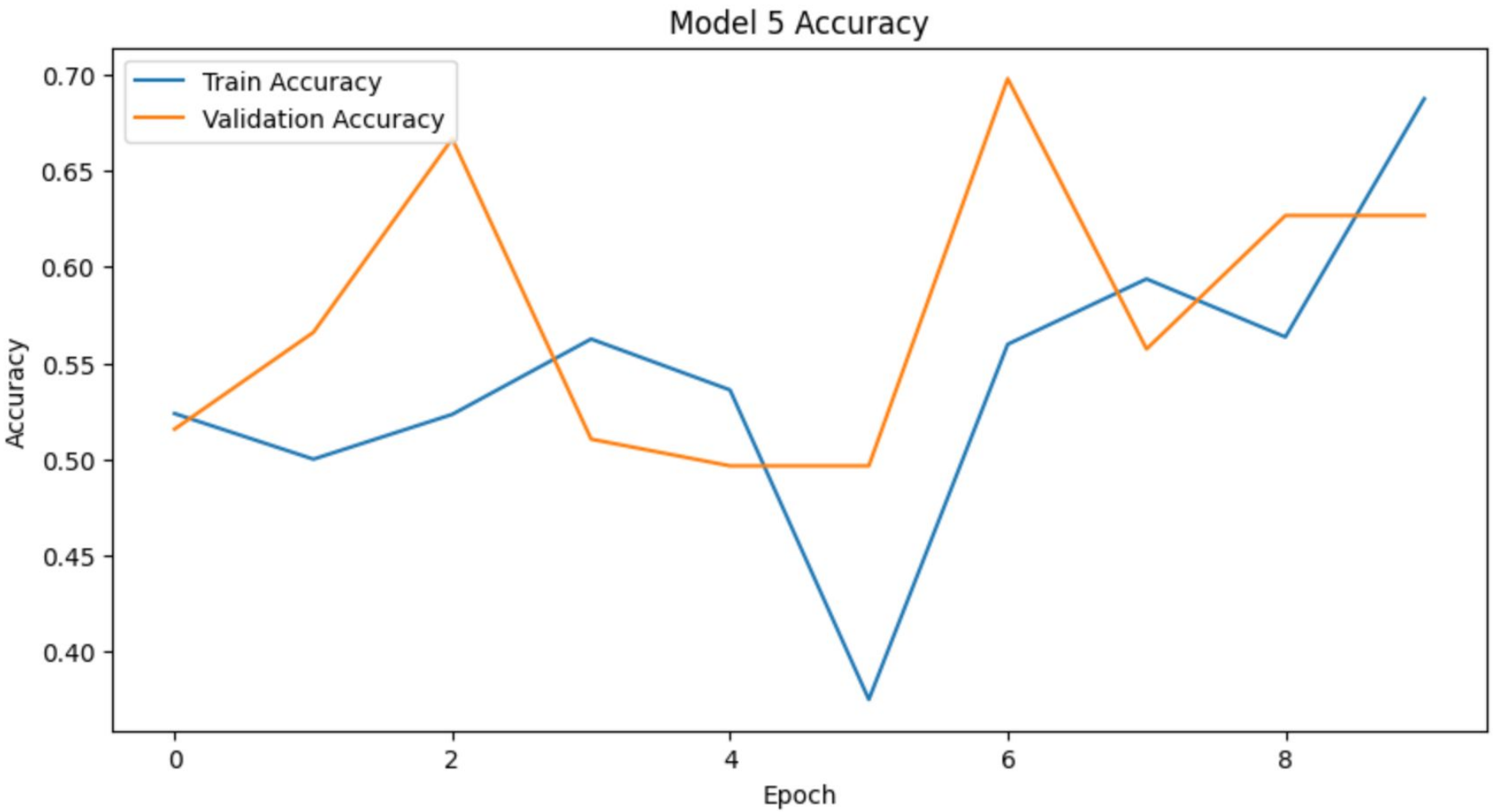
- **Model 3 is failing to learn effectively and is severely overfitting to the training data. Even when it achieves high accuracy on the training set (sometimes 100%), it cannot generalize those learnings to the validation set. This model is not suitable for deployment as it cannot make reliable predictions on new, unseen images.**

Model 4: VGG16 (Base + FFNN+ Data Augmentation)



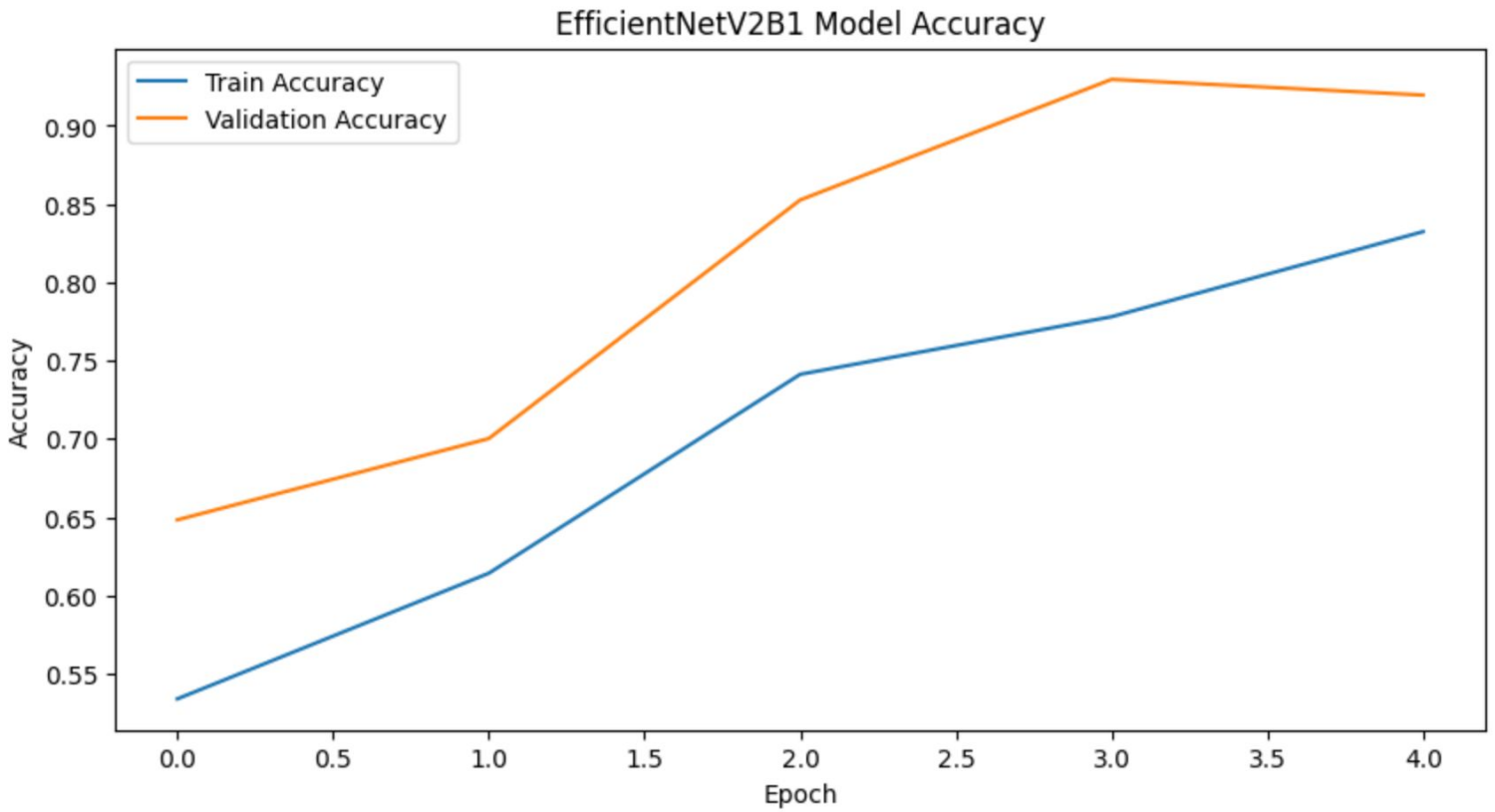
- **Model 4 learned effectively from the training data and maintained that strong performance on the validation set, demonstrating its robustness and generalization capabilities. But still not too good. It makes mistakes.**

Model 5: ResNet50



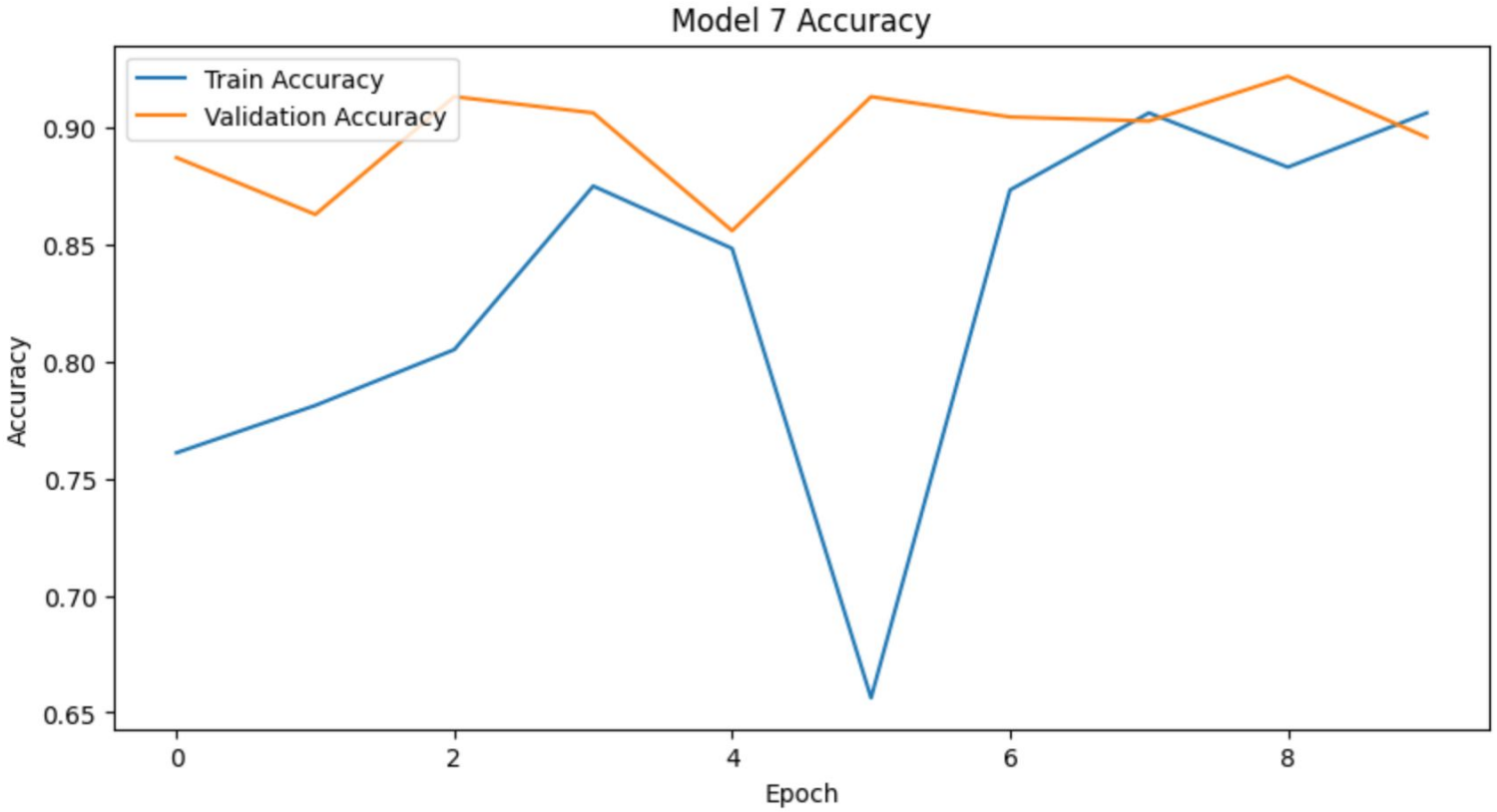
ResNet50 performance is only slightly better than a random guess (50%) and significantly worse than the best-performing VGG-16 model.

Model 6: Efficient Net



- This model achieved the highest accuracy (~91.96%) and should be considered if maximum accuracy is the sole priority and computational resources are less of a constraint.

Model 7: Mobile Net



- Model 7 Accuracy' graph shows a very promising performance for the MobileNetV2 Base model. It quickly learns to classify images with high accuracy and maintains strong performance on unseen data, making it a robust choice for the MonReader project.

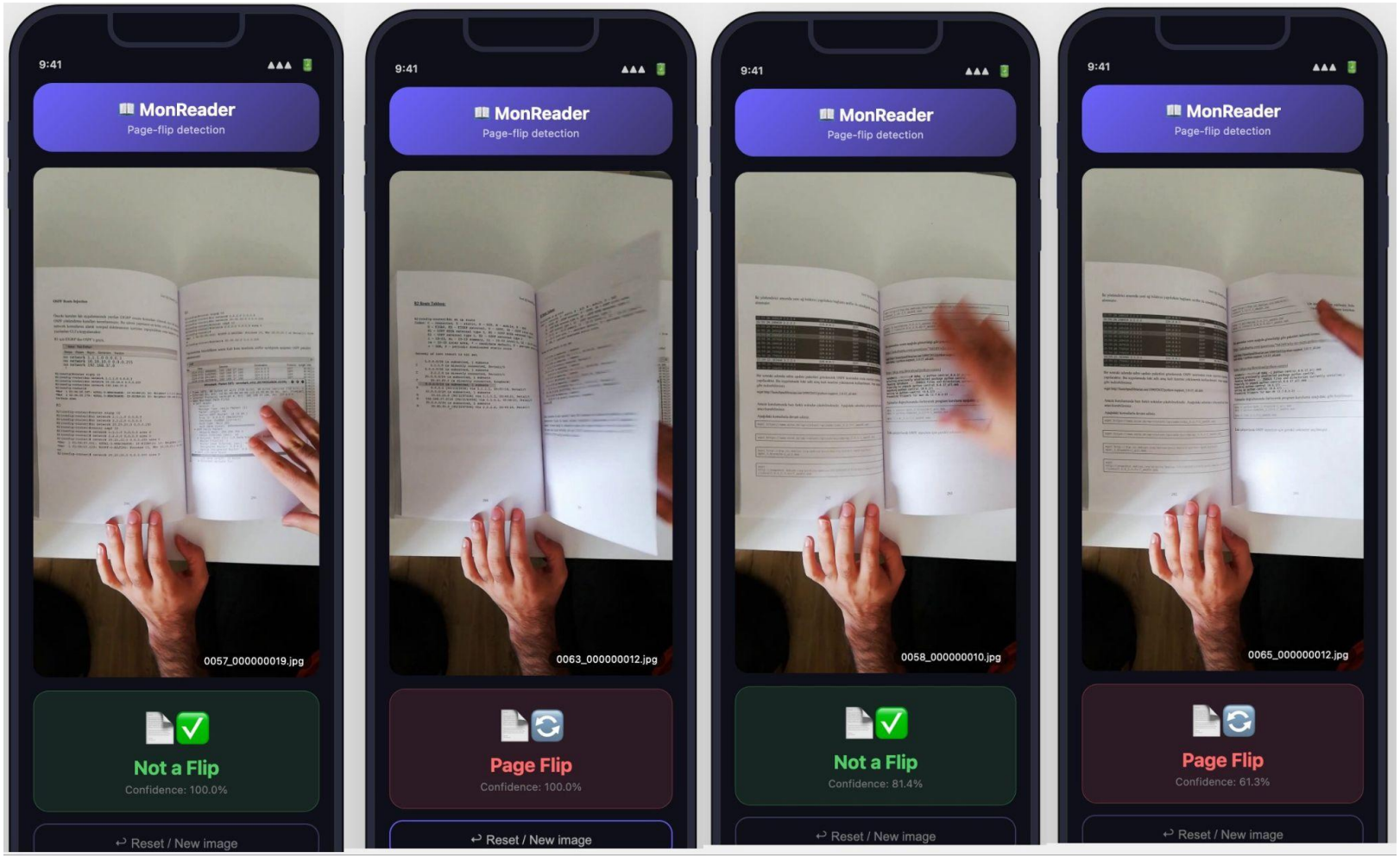
Model Comparison

Model	Accuracy	Recall	Precision	F1 Score	Size (MBs)
Simple CNN	0.514214	0.514214	0.264416	0.349245	1.57
VGG-16 (Base)	0.92893	0.92893	0.928953	0.928918	56.13
VGG-16 (Base+FFNN)	0.507107	0.507107	0.491735	0.422509	65.16
VGG-16 (Base+FFNN+Data Aug)	0.59464	0.59464	0.713517	0.519776	65.16
ResNet50 (Base)	0.628141	0.628141	0.685376	0.602816	90.36
EfficientNetB0 (Base)	0.919598	0.919598	0.924697	0.91949	30.27
🏆MobileNetV2 (Base)	0.896147	0.896147	0.903742	0.895879	8.85

Model Choice

Prioritize Model 7 (MobileNetV2) for Deployment: Given its strong performance (accuracy of ~89.61%) coupled with its lightweight architecture, Model 7 is the primary candidate for deployment, especially for resource-constrained mobile environments in the MonReader project. Its efficiency makes it ideal for on-device inference without significant compromise on accuracy.

Demo



- Monreader project deployed at: https://huggingface.co/spaces/dcsamuel/monreaderview
- Restart spaces, Make sure backend at https://huggingface.co/spaces/dcsamuel/monreader is running to view

Tools used

- Language: Python
- Deep Learning, TensorFlow, Keras
- Data Processing: Numpy, Pandas
- Visualisation: Matplotlib, Seaborn
- Computer Vision: OpenCV
- Evaluation: Scikit-learn

Future improvements

- **Further Investigation for Model 5 (ResNet50):** The ResNet50 base did not perform as expected. To improve its performance, consider:
  - **Fine-tuning:** Unfreeze the top layers of the ResNet50 base and fine-tune them with a very small learning rate along with the custom classification head.
  - **More Complex Head:** Experiment with different numbers of dense layers, dropout rates, and activation functions in the classification head.
- **Robustness Testing for Deployment:** Regardless of the chosen model, conduct extensive robustness testing with diverse real-world images that the MonReader application might encounter. This includes variations in lighting, background clutter, paper types, and page orientations to ensure real-world applicability.
- **Performance Monitoring in Production:** Implement continuous monitoring of the deployed model's performance. Track key metrics (accuracy, false positives/negatives) to detect performance degradation over time and inform retraining needs to maintain optimal performance.

Thank You

- Thank you for your time
- Contact: Samuel Mugisha DC
- Linkedin:samuelmugishadc
- Github: samuelmugisha